OASIS: Supplementary Materials

Author Names Redacted for Review

Affiliation Redacted
redacted@review.org

This document collects supplementary materials for the paper "OASIS: Repairing Stale Optimizer Statistics in the Feedback Nullspace." It provides detailed scope and error-budget positioning (§ A), including the marginal-to-joint composition boundary, model architecture and instrumentation details (§ B), baseline fairness discussion and QuickSel-H adaptation (§ C), the PostgreSQL-style statistics-format adapter and conversion protocol (§ D), predicate-coverage sensitivity analysis (§ E), observation aggregation study details (§ F), PostgreSQL planner-only statistics-injection protocol details (§ G), OOD drift realism protocol (§ H), benchmark-inspired DML trace and public production-telemetry trace protocols (§ I–J), feedback-budget, feedback-noise, and deployment-safety diagnostic protocols (§ K), Stage-2 calibration, estimator-agnostic prior-source swap, and feedback-locality details (§ L), and detailed overhead breakdowns (§ M).

1 Scope and Error Budget

OASIS targets *single-column histogram staleness*. We explicitly exclude from scope:

- **Multi-column correlation errors**: Join cardinality misestimation caused by correlated column predicates requires multi-dimensional statistics (e.g., mvdists, DNF-based sketches, or learned joint models). OASIS operates on one-dimensional marginals and is complementary to multi-column methods.
- **Physical design errors**: Missing indexes, suboptimal partitioning, or incorrect materialized views are outside the statistics layer.
- **Cost-model calibration**: Even with correct selectivities, the optimizer's cost model may mispredict operator runtimes. OASIS improves selectivity estimates but does not modify the cost model.
- **Data distribution shifts beyond drift**: Fundamental schema changes (column additions, type changes) or massive bulk loads that alter the distribution shape beyond what a compound drift operator captures are out of scope; these scenarios require a fresh `ANALYZE`.

Error budget decomposition. In a production optimizer, the total cardinality estimation error decomposes as:

$$\begin{aligned}\text{Total Error} = &\underbrace{\text{Single-Column Error}}_{\text{OASIS target}} + \underbrace{\text{Correlation Error}}_{\text{multi-column methods}} \\ &+ \underbrace{\text{Cost-Model Error}}_{\text{calibration}} + \underbrace{\text{Physical Design Error}}_{\text{indexing/partitioning}}.\end{aligned}$$

OASIS addresses the first term. Even perfect single-column statistics do not eliminate the remaining terms, but accurate marginals are a necessary prerequisite: learned multi-column estimators (e.g., copula-based models like CoLSE [6]) combine one-dimensional marginals with a learned dependence structure, making single-column statistics quality a foundational concern.

Marginal-to-joint composition boundary. The multi-column composition experiments in the main paper should be interpreted as downstream use of corrected marginals, not as evidence that OASIS estimates correlations. The independence estimator uses the maximum-entropy joint distribution implied by the supplied marginals, while the Gaussian-copula estimator receives a fixed dependence structure. In this setting, OASIS-noProj marginals improve the joint estimator only modestly and inconsistently because they are not always feedback-consistent on the current observation window. OASIS is therefore the deployable marginal: it starts from the learned OASIS-noProj marginal, then applies hard feedback-consistency projection before the marginal is passed to the downstream multi-column estimator. The core contribution remains single-column statistics correction.

2 Model and Instrumentation Details

This appendix collects implementation details trimmed from the main paper for page budget reasons.

2.1 Imputation-Layer Architecture

The imputation layer (Stage 1) is a compact *permutation-invariant set encoder* over the stale-quantile and feedback tokens. It is built so the same model consumes any feedback-window size and is invariant to observation order:

1. **Tokenization**: the normalized stale quantile boundaries, lightweight metadata, and up to K recent feedback observations (predicate type, boundary, estimated and actual selectivity) are embedded into a common token space, with a validity mask for unused slots.
2. **Set encoding**: the feedback tokens are aggregated by a permutation-invariant encoder (self-attention over the unordered window followed by masked pooling), so the representation is invariant to observation order and handles a variable number of observations.
3. **Residual head**: the pooled representation, fused with the encoded prior, predicts a correction delta $\boldsymbol{\delta}$ to the interior quantile boundaries; the output is clamped and monotonized to a valid CDF, so the model learns drift-induced deviations rather than absolute boundaries.

Training objective. Unlike a reconstruction-trained prior, the model is trained on the composite optimizer-facing objective of the main paper: the held-out future-predicate log-Q-error of the *projected* marginal, a FactorJoin bin-mass join-regret term, an in-window feedback-consistency residual, and a light reconstruction regularizer toward the post-drift marginal. Training uses a sparse-to-dense feedback curriculum and a domain-randomized mixture of drift families (batch loads, range/date shifts, skew evolution, outlier bursts, multimodal and seasonal drift) so the completion generalizes across feedback budgets and drift mechanisms. The model is trained offline in PyTorch with Adam and reused across columns without per-table retuning. The main-paper ablation shows that applying the projection inside the training loop is inessential—post-hoc projection performs identically—so the contribution is the training objective, not in-loop calibration.

2.2 Per-Conjunct Operator Instrumentation

Filter and scan-filter-project operators are augmented with per-conjunct selectivity counters. During execution, whenever a batch of rows is processed through a compound predicate $\phi_1 \wedge \phi_2 \wedge \cdots \wedge \phi_m$, each conjunct ϕ_j is evaluated independently on the full input batch:

$$n_j^{\text{in}} \mathrel{+}= |\text{batch}|, \qquad n_j^{\text{out}} \mathrel{+}= |\{r \in \text{batch} : \phi_j(r)\}|.$$

The final intersection of all conjunct results produces the same output as the original compound filter while additionally recording per-conjunct independent selectivities $s_j^* = n_j^{\text{out}}/n_j^{\text{in}}$. Upon operator completion, these counters are written to the engine's existing runtime metrics store. In the deployment model used by OASIS, the collector simply piggybacks on predicate evaluation already being performed by the engine, flushes a structured record to disk, and emits at most one observation tuple per filter conjunct. As a result, each query writes at most as many observation records as it has conjunctive filter clauses.

Algorithm 1 Per-Conjunct Operator Instrumentation

Require: Compound filter $\Phi = \phi_1 \wedge \cdots \wedge \phi_m$, input batch B
Ensure: Filtered output B', counters $n_j^{\text{in}}, n_j^{\text{out}}$

mask $\leftarrow$ [**true**]$^{|B|}$
for $j = 1$ **to** m **do**
    result$_j \leftarrow \phi_j(B)$
    $n_j^{\text{in}} \mathrel{+}= |B|$; $n_j^{\text{out}} \mathrel{+}= |\text{result}_j|$
    mask $\leftarrow$ mask $\wedge$ result$_j$
end for
return B[mask]

// On operator completion, emit per-conjunct metrics:
for $j = 1$ **to** m **do**
    EmitMetric(conjunct_j, n_j^{in}, n_j^{out})
end for

2.3 QuickSel-H Adaptation Details

The QuickSel system [5] is a selectivity learner that fits a mixture-model density to satisfy feedback constraints. Because QuickSel outputs point selectivity estimates rather than histogram boundaries, we evaluate a clearly labeled **QuickSel-H** adaptation that bridges the gap between selectivity learning and histogram correction. The adaptation performs three steps:

1. **Mixture-model fitting**: Given the same stale prior H_0 and observation window $\mathcal{O}$ of size K=16, QuickSel-H fits a Gaussian mixture model with M=5 components whose CDF is constrained to match observed selectivities at the predicate values in $\mathcal{O}$. The optimization uses the same relative-entropy objective as the original QuickSel, solved via iterated scaling.
2. **Quantile extraction**: From the fitted mixture CDF, we extract the $B-1$=9 quantile values at the equi-depth levels used by all other methods. This ensures QuickSel-H outputs on the same histogram grid.
3. **Output interface**: The extracted quantiles are passed through the same monotonic projection as OASIS and returned as the corrected histogram.

Why this adaptation is necessary. QuickSel's native output is a density estimate over the full domain, not a fixed-resolution histogram consumable by the CBO. To compare under a matched output interface, we must extract quantiles from this density. The extraction step introduces a minor information bottleneck (compressing a rich density into $B-1$ boundaries), but this is identical to the bottleneck all other methods face and does not fundamentally change QuickSel's correction mechanism. We do not claim QuickSel-H is a line-by-line reproduction of the original system; it is a faithful adaptation that preserves QuickSel's core density-fitting approach while conforming to the common output format required by our evaluation protocol.

3 Baseline Fairness Discussion

All feedback-driven methods compared in the main paper consume the same observation window of size K=16 and emit corrected quantiles on the same B=10 equi-depth histogram grid. No method receives additional information or computational budget beyond this shared interface.

STHoles. We implement the hierarchical refinement algorithm described in [2]. STHoles builds a tree of bucket refinements from feedback. We initialize it from the stale prior and allow up to K refinement steps per correction instance, using the same observation tuples provided to OASIS. The output quantiles are extracted from the refined tree on the standard $B-1$ grid.

ISOMER. We implement the consistency-based histogram adjustment from [4]. ISOMER adjusts bucket frequencies to satisfy observed selectivity constraints while minimizing KL divergence from the prior. We provide the same K=16 observation tuples and extract quantiles on the standard grid.

QuickSel-H. See § 2 for the detailed QuickSel-H adaptation.

Simple heuristics. LinInterp and FeedAvg consume the same observation window but use fixed aggregation rules (blend factor 0.5 and exponential moving average, respectively). They are included to demonstrate that learned correction is necessary, not merely that any feedback-driven approach helps.

No method receives per-table or per-distribution tuning; all use default hyperparameters. Results are averaged over three independent training seeds (42, 123, 456) with 95% confidence intervals.

4 PostgreSQL-Style Statistics-Format Adapter

4.1 Motivation

OASIS corrects a canonical full-distribution view of a column. This is straightforward when a DBMS exposes a self-contained histogram, but many production systems do not. Instead, the optimizer's statistics object often couples the histogram with auxiliary summaries whose probability mass must remain consistent after any update.

- **PostgreSQL**: Maintains a Most Common Values (MCV) list alongside a residual histogram that excludes MCV entries.
- **Oracle**: Uses top-frequency values together with height-balanced histograms.
- **SQL Server**: Employs a density vector together with histogram steps.

This coupling is the main adapter boundary for histogram-correction methods: *correcting the histogram alone* is often insufficient because the stored statistics object is not a standalone histogram. OASIS addresses this with a *statistics-format conversion interface* that translates a native statistics layout into a canonical full distribution, applies OASIS there, and translates the corrected result back.

We instantiate the interface on PostgreSQL's MCV case because it is both practically important and representative of tightly coupled statistics architectures. The design isolates engine-specific adapter logic, but this paper validates only the PostgreSQL-style MCV+histogram instantiation.

4.2 Three-Phase Conversion Interface

The interface follows three phases:

1. **Reconstruction**: Convert engine-native statistics → canonical full distribution
2. **Correction**: Apply OASIS on the canonical representation
3. **Decomposition**: Convert corrected full distribution → engine-native statistics

This design decouples OASIS's correction logic from PostgreSQL-specific storage fields. Other engines would require separate adapter implementations and validation.

4.3 PostgreSQL Instantiation

4.4 Problem Formulation

Given PostgreSQL statistics:
- MCV list: $\mathcal{M} = \{(v_1, f_1), (v_2, f_2), \ldots, (v_k, f_k)\}$
- Residual histogram bounds: $\mathcal{H} = \{u_0, u_1, \ldots, u_m\}$ where adjacent values define m equi-depth residual intervals
- Null fraction: f_{null}
- Non-null invariant: $\sum_{i=1}^{k} f_i + p_{\text{res}} = 1 - f_{\text{null}}$

Construct a canonical full distribution $\mathcal{F}$ that exposes the complete non-null column distribution to OASIS.

4.5 Phase 1: Reconstruction

Algorithm 2 Reconstruct Canonical Full Distribution from PostgreSQL Statistics

Require: MCV list $\mathcal{M}$, residual bounds $\mathcal{H} = \{u_0, \ldots, u_m\}$, null fraction f_{null}
Ensure: Canonical full distribution $\mathcal{F} = (\mathcal{S}, \mathcal{R})$
1: Initialize $\mathcal{S} \leftarrow \emptyset$, $\mathcal{R} \leftarrow \emptyset$
2: $p_{\text{res}} \leftarrow 1 - f_{\text{null}} - \sum_{(v_i, f_i) \in \mathcal{M}} f_i$
3: **for** each $(v_i, f_i) \in \mathcal{M}$ **do**
4: Add singleton bucket (v_i, f_i) to $\mathcal{S}$
5: **end for**
6: **for** $j = 1$ **to** m **do**
7: Add range bucket $(u_{j-1}, u_j, p_{\text{res}}/m)$ to $\mathcal{R}$
8: **end for**
9: Validate: $\sum_{s \in \mathcal{S}} s.\text{freq} + \sum_{r \in \mathcal{R}} r.\text{freq} = 1 - f_{\text{null}}$ **return** $\mathcal{F} = (\mathcal{S}, \mathcal{R})$

PostgreSQL stores histogram *boundary values*, not explicit per-bucket frequencies. The conversion interface therefore interprets the residual histogram as m equal-depth intervals carrying equal shares of the residual non-MCV mass. Before tensorization, the interface evaluates the CDF of $\mathcal{F}$ at OASIS's fixed quantile levels, yielding the canonical quantile vector consumed by the model.

4.6 Phase 2: OASIS Correction

Once reconstructed, the canonical full distribution is passed unchanged to the standard OASIS correction pipeline. The important systems property is that the model itself remains ignorant of the source database format; all engine-specific handling is localized to the conversion interface.

4.7 Phase 3: Decomposition

Given a corrected full distribution $\mathcal{F}' = (\mathcal{S}', \mathcal{R}')$, reconstruct PostgreSQL-compatible statistics:

– new MCV list: $\mathcal{M}'$
– new residual histogram bounds: $\mathcal{H}'$
– invariant: $\sum_i f'_i + p'_{\text{res}} = 1 - f_{\text{null}}$

Algorithm 3 Decompose Canonical Full Distribution to PostgreSQL Statistics

Require: Full distribution $\mathcal{F}' = (\mathcal{S}', \mathcal{R}')$, MCV threshold τ, MCV limit K, target histogram size $m{+}1$
Ensure: MCV list $\mathcal{M}'$, residual histogram bounds $\mathcal{H}'$
Step 1: Extract MCV candidates
$\mathcal{C} \leftarrow \{(v, f) \mid (v, f) \in \mathcal{S}' \text{ and } f \geq \tau\}$
Sort $\mathcal{C}$ by frequency (descending)
$\mathcal{M}' \leftarrow$ top K entries from $\mathcal{C}$
Step 2: Build residual distribution
$V_{\text{mcv}} \leftarrow \{v \mid (v, f) \in \mathcal{M}'\}$
Construct residual mixture from all range buckets in $\mathcal{R}'$ and all singletons in $\mathcal{S}'$ whose value is not in V_{mcv}
Step 3: Resample equi-depth histogram bounds
for $j = 0$ **to** m **do**
    Set u_j to the residual quantile at level j/m
end for
$\mathcal{H}' \leftarrow \{u_0, \ldots, u_m\}$
Validate: $\sum_{(v,f)\in\mathcal{M}'} f + p'_{\text{res}} = 1 - f_{\text{null}}$ **return** $\mathcal{M}'$, $\mathcal{H}'$

4.8 MCV Selection Policy

The MCV threshold τ and limit K control the final decomposition:
– **High threshold**: keeps the MCV list compact but pushes more probability mass into the residual histogram
– **Low threshold**: retains more explicit singleton masses as MCV entries
– **PostgreSQL default-like regime**: $\tau \approx 0.01$, $K = 100$

This threshold is a decomposition policy choice, not part of OASIS's core correction model. In our validation, experiments 1–3 remove threshold effects by using threshold-free round trips, while experiment 4 isolates the effect of τ_{MCV} after a simulated correction.

4.9 Adapter Boundary Beyond PostgreSQL

The PostgreSQL instantiation is only one instance of the conversion pattern. The same interface idea can be implemented for other engines that rely on histograms to very different degrees, but those implementations are outside the experimental scope of this paper:
– **Self-contained histograms** (MySQL, MariaDB): direct pass-through; conversion degenerates to a near-identity mapping.
– **MCV-coupled statistics** (PostgreSQL, Oracle): reconstruct from explicit frequent-value summaries plus residual histograms, then decompose back.

– **Hybrid summaries** (SQL Server): reconstruct from histogram steps plus density summaries, apply OASIS, then recompute auxiliary summaries.

The systems contribution here is to expose and validate this adapter boundary for a PostgreSQL-style statistics object, not to claim that all DBMS formats are already supported.

4.10 Consistency Guarantees

4.11 Probability Mass Conservation

Theorem 1 (Probability Conservation). *If the source statistics satisfy the non-null invariant* $\sum_i f_i + p_{\text{res}} = 1 - f_{\text{null}}$*, then:*

1. *reconstruction produces a canonical full distribution with the same non-null mass;*
2. *decomposition emits engine-native statistics whose MCV mass plus residual mass again equals* $1 - f_{\text{null}}$*.*

Proof. Reconstruction assigns all explicit MCV mass to singleton buckets and all remaining non-null mass to residual intervals by construction. Decomposition selects an explicit MCV set and resamples the remainder as residual histogram bounds, so the output MCV mass and residual mass partition the same corrected non-null mass. Therefore the invariant is preserved.

4.12 Approximate Idempotence

Definition 1 (Approximate Idempotence). *Let*

$$(\mathcal{M}', \mathcal{H}') = \text{Decompose}(\text{Reconstruct}(\mathcal{M}, \mathcal{H}), \tau, K).$$

The conversion interface is ϵ*-idempotent if the round trip preserves total non-null mass within* ϵ *and preserves the residual quantile function within* ϵ *over a fixed evaluation grid.*

Perfect idempotence need not hold under thresholded decomposition because:

– the MCV threshold may change which values remain explicit,
– low-frequency singleton mass may be folded back into the residual distribution,
– floating-point rounding accumulates.

Our threshold-free validation shows exact total-mass preservation and machine-precision residual-quantile fidelity for the tested PostgreSQL-style statistics.

4.13 Implementation Interface

We provide a Python reference interface that exposes the conversion pattern explicitly:

Listing 1.1: Statistics-Format Conversion Interface (PostgreSQL Instance)

```python
class PostgreSQLStatisticsAdapter:
    def to_full_histogram(
        self,
        pg_stats: PostgreSQLStatistics
    ) -> FullHistogram:
        """Phase 1: Reconstruction"""
        pass

    def from_full_histogram(
        self,
        full_hist: FullHistogram
    ) -> PostgreSQLStatistics:
        """Phase 3: Decomposition"""
        pass

    def round_trip_test(
        self,
        pg_stats: PostgreSQLStatistics
    ) -> Tuple[bool, str]:
        """Validate round-trip fidelity"""
        pass
```

The full reference interface is available in `appendix/mcv_adapter_interface.py`. This appendix is the supplementary counterpart of the compact PostgreSQL-style adapter discussion in the main text and is intended to be kept numerically consistent with `experiments/results/mcv_validation_results.json`.

4.14 Performance Considerations

4.15 Time Complexity

- **Reconstruction**: $O(k + n)$ where $k = |\mathcal{M}|$ and $n = |\mathcal{H}|$
- **Decomposition**: dominated by MCV candidate sorting and residual quantile sampling
- **Total overhead**: negligible relative to correction and well within planning-time constraints in our PostgreSQL validation

4.16 Space Complexity

The canonical full distribution is temporary and small:

$$\text{Space}(\mathcal{F}) = \text{Space}(\mathcal{M}) + \text{Space}(\mathcal{H}) + O(1).$$

In practice this overhead is acceptable because statistics objects are compact, the canonical distribution exists only during correction, and no persistent storage expansion is required.

4.17 Validation Summary

We validate the PostgreSQL instantiation of the conversion interface on synthetic PostgreSQL-style statistics spanning uniform, normal, Zipfian, and bimodal distributions.

Table 1: PostgreSQL Statistics-Format Interface Validation Summary

Metric	**Result**
Round-trip total mass error	0
Residual quantile MAE (mean)	2.4×10^{-17}
Residual quantile MAE (max)	9.2×10^{-17}
Selectivity MAE	1.1×10^{-4}
Selectivity P99	3.8×10^{-3}
Default-like overhead (100 MCV, 10–20 bins)	0.066–0.073 ms
Worst tested overhead (50 MCV, 50 bins)	1.03 ms

These results support three claims. First, the conversion interface is exact in mass preservation and essentially exact in residual-quantile reconstruction under threshold-free round trips. Second, it introduces negligible estimation bias and low latency on PostgreSQL-like settings. Third, threshold sensitivity is graceful: increasing τ_{MCV} shrinks the MCV list and gradually increases selectivity error, but does not compromise mass conservation.

4.18 Takeaway

The main value of this adapter is that it keeps histogram correction separate from native statistics storage details. OASIS corrects the canonical full-distribution view; the PostgreSQL-style adapter reconstructs and decomposes the native MCV+histogram object around that view. Extending the same pattern to other DBMSs is a design direction rather than an evaluated claim in this paper.

5 Predicate-Coverage Sensitivity Analysis

OASIS correction quality depends on the observation window covering the value range relevant to future predicates. The dependence on *how much* of the domain the feedback constrains is exactly the nullspace effect the main paper characterizes: the gain concentrates on predicates far from any feedback (the feedback-locality diagnostic, § 12) and the sparse-feedback regime is where the learned completion helps most on a real planner (the PostgreSQL budget sweep in the main paper). Where feedback is too sparse to constrain the queried range, the residual-gated Router falls back to the maximum-entropy projection, so accuracy degrades gracefully rather than catastrophically.

6 Observation Aggregation Study

The imputation layer aggregates the unordered feedback window with a permutation-invariant set encoder, so that the same model consumes any window size and is order-independent. The architecture is deliberately secondary to the training objective: the main paper's ablation (Section on the composite-objective

ablation) holds the model and projection fixed and shows that a reconstruction-only objective is worse than the classical priors at every feedback budget, while the composite optimizer-facing objective produces the gain. In other words, the decisive design choice is *what the prior is trained to optimize*, not the pooling mechanism.

7 PostgreSQL Planner-Only Statistics-Injection Protocol

The planner-only evaluation protocol uses PostgreSQL's `EXPLAIN` (without `ANALYZE`) to validate the statistics→estimated-cardinality→plan-shape path. `EXPLAIN` invokes PostgreSQL's optimizer and cost model, but it does not execute the query. The protocol therefore provides optimizer-facing evidence without making wall-clock runtime claims.

Protocol steps.

1. Create a synthetic PostgreSQL schema with a fact table and dimension table. The fact table stores the current post-drift data.
2. Generate stale and fresh single-column distributions for `fact.x`; build ISOMER, OASIS-noProj, OASIS, and Hybrid corrected marginal statistics from the same feedback window.
3. For each statistics state, create an analyzed one-column source table and copy its typed `pg_statistic` row into the target `fact.x` statistics entry. This avoids hand-writing PostgreSQL's `anyarray` catalog fields.
4. For each test query, obtain true cardinality using `COUNT(*)` and obtain PostgreSQL estimated rows and plan shape using `EXPLAIN (FORMAT JSON)`.
5. Compare row-estimation Q-error, plan-shape match to the fresh-statistics planner, recovery of stale/fresh plan disagreements, and new deviations from fresh when stale already matched fresh.

Results. The formal batch covers 12 configurations: two drift directions (`left_shift` and `right_shift`), two fact-table sizes (100K and 200K rows), and three random seeds. Each configuration evaluates 84 scan/join query templates under each statistics state. No query runtime is measured.

- Stale statistics produce aggregate row Q-error 27.768 and match the fresh-statistics plan shape on 56.1% of queries.
- OASIS-noProj reduces row Q-error to 3.703 and recovers 82.2% of stale/fresh plan-shape disagreements, but introduces new plan deviations in 2.3% of cases where stale statistics already matched fresh.
- OASIS reduces row Q-error to 2.362, matches the fresh-statistics plan shape on 95.9% of queries, recovers 92.1% of stale/fresh plan disagreements, and reduces new plan deviations to 1.1%.
- Across configurations, OASIS has mean row Q-error 2.377 ± 0.269, close to ISOMER's 2.390 ± 0.286 and below OASIS-noProj's 3.894 ± 1.252.

This protocol validates the causal chain

$$\text{statistics correction} \rightarrow \text{estimated rows} \rightarrow \text{cost re-estimation} \rightarrow \text{plan shape}$$

inside a real PostgreSQL optimizer, while intentionally avoiding runtime claims that would depend on executor, cache, and system effects outside the single-column statistics layer.

Representative plan-shape changes from the batch show the mechanism the planner-only experiment validates: stale statistics can move estimated rows enough to select a different root operator or join strategy, while OASIS moves the estimated rows and plan shape back toward the fresh-statistics planner. These plan changes are not used as runtime evidence.

8 OOD Drift Realism Protocol

The OOD drift realism suite evaluates the same OASIS checkpoint used in the main synthetic experiments. The checkpoint is trained only on the compound drift generator; no samples from the OOD families are used for retraining or hyperparameter tuning. Each OOD case follows the same causal protocol as the main synthetic suite: capture stale quantiles, interleave drift steps with 16 feedback observations, compute final post-drift quantiles, and evaluate future selectivity estimates against the final distribution.

The six OOD families are deployment-inspired patterns implemented in `extended_drift_generators.py`: batch loading, range shift, skew evolution, outlier burst, multimodal drift, and seasonal/mixed drift. The reported table uses 128 held-out cases per family, $B = 10$ histogram buckets, and 64 future CDF probe points per case. This experiment is not intended to prove that the simulator is a production DML trace; it tests whether OASIS-noProj and OASIS remain useful when the drift mechanism differs from the training generator.

9 Benchmark-Inspired DML Trace Protocol

The trace-grounded sanity check is not a production-log experiment. It constructs explicit DML event streams whose table/column stories are anchored to analytical benchmark maintenance patterns: append-heavy sales dates, promotion price revisions, inventory restocking, returns and cancellations, customer segment churn, and seasonal mixed maintenance. Values are normalized to the unit interval so that the same single-column correction interface is used as in the synthetic and OOD experiments.

Each trace case begins with a stale histogram, then executes 16 DML steps. Each step emits concrete insert, update, and delete counts and then records one predicate feedback observation against the current post-step table. The reported run uses 96 cases per trace, $B = 10$ histogram buckets, and 64 future CDF probe

Table 2: Benchmark-inspired DML trace sanity check. Each trace emits explicit insert/update/delete events anchored to an analytical benchmark table/column pattern. Values are geometric mean selectivity Q-error; no production trace or query runtime is used.

Trace	DML mix I/U/D	Growth	Stale	ISOMER	OASIS-noProj	OASIS	Soft	Hybrid	Aggressive	Fresh
Sales append	96/0/4	+79.9%	1.529	1.128	1.133	**1.114**	1.181	1.135	1.138	1.000
Promotion price	30/70/0	+33.9%	1.025	**1.018**	1.027	1.020	1.019	1.019	1.018	1.000
Inventory restock	19/75/6	+19.8%	1.222	**1.042**	1.127	1.046	1.054	1.047	1.050	1.000
Returns/cancel	38/17/45	-3.6%	2.047	1.163	1.161	**1.118**	1.187	1.170	1.172	1.000
Customer churn	36/62/2	+41.2%	1.172	**1.057**	1.159	1.079	1.084	1.061	1.063	1.000
Seasonal mixed	58/38/4	+50.8%	1.029	1.020	1.041	1.021	1.019	**1.019**	1.020	1.000

points per case. The same OASIS checkpoint trained on compound drift is used unchanged; no trace-specific retraining or hyperparameter tuning is performed.

The table should be interpreted as a realism sanity check rather than a claim that the traces match a particular deployment. Its main role is to expose a useful boundary: OASIS and Hybrid consistently improve over stale statistics, while OASIS-noProj can regress on mild update-heavy traces where feedback consistency is more valuable than learned global extrapolation.

10 Public Production-Telemetry Trace Protocol

The public production-telemetry case study uses the NASA Kennedy Space Center HTTP access log distributed by the Internet Traffic Archive [3]. The trace contains real web-server requests from July 1995 and is a standard workload-characterization data source [1]. We use it because the project does not include private production DBMS workload logs. Accordingly, the experiment should be read as a trace-grounded append-only analytics-table case study, not as evidence from a live database deployment or a private query workload.

The experiment parses the first 750,000 valid requests from the July trace (1995-07-01 00:00:01 to 1995-07-11 13:12:57, UTC-04:00), skipping five malformed lines. Each HTTP request appends one event. We derive three normalized single-column signals from real request fields: *reply bytes* is $\log(1 + \text{bytes})$ scaled by the 99.5th percentile, *time of day* is seconds since midnight divided by 86,400, and *path length* is $\log(1 + \text{request-path length})$ scaled by the 99.5th percentile. The resulting values use the same unit-interval marginal-statistics interface as the other OASIS experiments.

For each signal we draw 96 windows. A window builds stale statistics from 2,000 initial events, then performs 16 append-and-feedback steps with 1,000 later real events per step, for an 800% append growth relative to the stale prefix. The windows are stratified across the single public trace and may overlap; we therefore report this as an external-validity case study, not as an independent sample for significance testing. The feedback predicates are sampled from the current post-append table using the same single-column predicate mix as the other trace experiments. Future selectivity Q-error is evaluated on 64 predicate constants, half drawn from the held-out later requests in the same trace window and half

from the unit interval. The trained OASIS checkpoint, hard projection, Soft operator, Hybrid rule, and Router are used unchanged; there is no trace-specific retraining or hyperparameter tuning.

11 Feedback Budget, Feedback Noise, and Safety Diagnostics

The feedback-budget experiment truncates each held-out sample to the K most recent observations, with $K \in \{2, 4, 8, 16\}$. The trained OASIS checkpoint is not retrained for smaller budgets; instead, the truncated window is padded to the checkpoint's native 16-slot tensor width. ISOMER, OASIS, and Hybrid receive the same truncated observation window, so the comparison isolates the effect of feedback availability.

The feedback-noise experiment perturbs only the observed actual selectivities in the feedback window. It applies multiplicative log-normal noise with $\sigma \in \{0, 0.02, 0.05, 0.10\}$, keeps stale priors and fresh future selectivities fixed, and evaluates the same generator-driven optimizer-decision proxy used in the main paper. The reported diagnostic uses 64 held-out cases per drift intensity and one noise seed, because noisy projection constraints are more expensive than the clean-feedback setting.

Both diagnostics are optimizer-facing rather than runtime-facing. The scan and join decisions are selected from estimated selectivities under a fixed cost proxy, then scored by true proxy cost using the generated post-drift distribution. The purpose is to characterize deployment sensitivity, not to claim wall-clock speedups.

The deployment-safety table in the main paper is derived from cached outputs of four diagnostics: PostgreSQL planner-only batch summaries, feedback-budget sensitivity, feedback-noise robustness, and the benchmark-inspired DML trace sanity check. It does not introduce an oracle selector. Projection uses only the current feedback window, and Hybrid chooses among stale, ISOMER, OASIS-noProj, and OASIS by minimizing feedback-window residual. The table is generated by `experiments/deployment_safety_analysis.py`.

12 Stage-2 Calibration: Soft, Banded Projection, and Router

This section gives the protocol and full per-family results behind the main paper's finding that the Stage-2 calibration is regime-dependent.

12.1 Operators

Stage-2 operators are defined for any supplied marginal prior. In the fixed-prior comparison below, the supplied prior is the OASIS-noProj histogram; the

prior-source swap later in this section varies that input. All operators act on the same interval partition induced by the active feedback predicates. *Hard projection* (the default exact Stage-2 operator) performs cyclic I-projections so that every active predicate's interval mass matches its observed selectivity exactly, dropping the oldest constraints when the active set becomes infeasible. *Soft projection* instead minimizes $\mathrm{KL}(p \,\|\, p_{\text{prior}}) + \lambda \sum_j w_j (A_j p - y_j)^2$ by mirror descent over cell masses, where A_j is the interval mask of feedback j and y_j its observed selectivity. We study three weightings: full-window (w_j=1), a fixed recent window (w_j=1 for the last k=8 observations, 0 otherwise), and *conflict-aware*. The conflict-aware weighting fits a hard reference distribution p_{ref} to the most recent k observations, scores each constraint by $\text{conflict}_j = |A_j p_{\text{ref}} - y_j|$, and sets $w_j = \exp(-(\text{conflict}_j/\tau)^2)$ with τ=0.03, so feedback contradicted by the recent observations is suppressed while old-but-consistent feedback is retained. *Banded projection* I-projects p_{prior} onto the bands $y_j \pm \kappa\sqrt{y_j(1-y_j)}$ (κ=0 recovers hard projection). In the fixed-prior comparison, *Router* selects, per case, the candidate with the lowest feedback-window residual among {stale, ISOMER, OASIS-noProj, OASIS, Soft}; in the prior-source swap, the raw supplied prior replaces OASIS-noProj in the same protocol. All operators and the diagnostic are implemented in `cdf_kll_ml_pipeline/modern_baselines.py` (`correct_soft_isomer`, `correct_band_isomer`) and `experiments/oasis_soft_projection_diagnostics.py`.

12.2 Evaluation Protocol

The single-column comparison reuses the cached compound-drift data and the fixed OASIS checkpoint; it does not retrain. For each case it generates held-out future predicates, estimates their selectivity from each Stage-2 output, and reports the geometric-mean Q-error, together with the feedback residual on the observation window, quantile MAE against fresh statistics, and an optimizer-proxy join regret. This future-predicate aggregation differs from the `single_column_projection` table in the main results (which reports a different per-case aggregation), so the numbers here are not directly comparable to that table and are presented as a self-contained operator comparison. The downstream and safety suites (OOD drift, DML trace, FactorJoin, composition family, optimizer proxy, PostgreSQL planner subset) reuse the same protocols documented in the preceding sections, with the soft/router methods added.

12.3 Single-Column and Optimizer-Proxy Results

Table 3 reports the operator comparison on the generator-driven optimizer-decision proxy. The ordering is stale (2.867) → ISOMER (1.664) → hard projection of the learned prior (1.401) → Soft (1.362) → Router (1.375, 92.9% join-optimal), approaching fresh. Soft has the lowest single-column Q-error but, as the main-paper ablation shows, the gain over hard projection is small and is not the source of OASIS's advantage; the Router recovers most of Soft's accuracy

Table 3: Generator-driven optimizer-decision proxy. Q-Error and regret are geometric means; regret is true proxy cost of the plan selected from estimated statistics divided by the true optimal proxy cost (1.0 is optimal). No query runtime is measured.

Method	Sel. QE	Join Regret	Join Opt.	Fresh Match	Risk Resolved
Stale	2.867	1.187	79.0%	79.0%	0.0%
ISOMER	1.664	1.063	88.8%	88.8%	54.3%
OASIS-noProj	1.565	1.042	90.9%	90.9%	66.1%
OASIS	1.401	1.029	92.8%	92.8%	74.5%
Soft	1.362	1.025	93.2%	93.2%	75.3%
Hybrid	1.437	1.034	92.2%	92.2%	70.8%
Aggressive	1.441	1.035	92.1%	92.1%	70.1%
Router	1.375	1.028	92.9%	92.9%	74.0%
Fresh	1.000	1.000	100.0%	100.0%	100.0%

Table 4: Stage-1 prior-source swap under a fixed Stage-2 calibration protocol. Each row supplies a different single-column marginal prior to Stage 2; lower Q-error and feedback residual are better.

Stage-1 source	Raw QE	+Hard QE	+Router QE	Router gain	Raw resid.	Router resid.
Stale prior	2.404	1.539	1.437	40.2%	0.1206	0.0383
LinInterp	2.116	1.508	1.417	33.0%	0.1072	0.0379
FeedAvg	2.474	1.544	1.437	41.9%	0.1256	0.0383
STHoles	1.572	1.436	1.394	11.3%	0.0542	0.0365
QuickSel-H	1.614	1.423	1.392	13.7%	0.0573	0.0347
LQM (learned QD)	1.607	1.454	1.425	11.3%	0.0471	0.0336
OASIS MLP	1.489	1.346	1.328	10.8%	0.0628	0.0354

while remaining deployment-safe and never selecting a higher-residual candidate. As a negative control, banded projection (relaxing the equality constraints to confidence bands) monotonically *worsens* single-column Q-error as the band widens, confirming that the gain comes from per-case routing rather than from relaxing the feedback constraints.

12.4 Estimator-Agnostic Prior-Source Swap

The main text also asks whether Stage 2 is merely an OASIS-MLP-specific patch. The prior-source swap uses the same cached drift cases and the same held-out future-predicate protocol as the Stage-2 comparison, but replaces the supplied marginal prior with six alternatives: stale statistics, LinInterp, FeedAvg, STHoles, QuickSel-H, and the OASIS MLP. Each source is evaluated raw, after hard projection initialized from that source, and after Router over {stale, ISOMER, raw prior, hard projection, Soft}.

Table 4 shows that the calibration layer is estimator-agnostic in aggregate. Hard projection improves every raw prior, and Router further improves every source-level row, reducing future Q-error by 10.8%–41.9% relative to the

Table 5: Feedback-locality diagnostic on the mixed predicate population. Held-out future predicates are binned by the directed Hausdorff distance from their endpoints to the nearest feedback endpoint in fresh-CDF space. Values are geometric-mean selectivity Q-error; ISOMER is projection from a stale start and OASIS is the learned prior plus hard projection. OASIS win is the per-predicate fraction on which OASIS beats ISOMER.

Locality	Preds	Stale	ISOMER	OASIS-noProj	OASIS	Fresh	OASIS vs ISOMER	OASIS win
near¡= 0.05	10556	2.097	1.424	1.389	1.295	1.000	+9.1%	64.2%
mid¡= 0.15	6911	2.494	1.561	1.473	1.335	1.000	+14.5%	63.2%
far¿ 0.15	3013	3.186	1.819	1.564	1.433	1.000	+21.2%	66.1%

corresponding raw marginal, with the final Router outputs clustered tightly (1.39–1.44). The OASIS prior is the strongest *raw* input (1.489, below STHoles 1.572, QuickSel-H 1.614, and the recent learned query-driven baseline LQM 1.607) and routes to the best final value (1.328, versus 1.425 for LQM). Fitting a flexible learned model to the feedback alone (LQM) therefore lands with the classical self-tuning histograms; the advantage comes from training the completion against optimizer error, while the reusable mechanism is the calibration layer that accepts several imperfect marginal sources.

12.5 Feedback-Locality Diagnostic

To see where the learned imputation helps after projection, we bin the held-out future predicates by directed Hausdorff distance from their endpoints to the nearest feedback endpoint in fresh-CDF space. This is the spatial signature of a nullspace completion: OASIS beats ISOMER in every locality bin, and the advantage *grows* with distance from feedback—+9.1%, +14.5%, and +21.2% in the near, mid, and far bins. The constraints determine the marginal near the observations, and the learned prior earns its advantage where they do not.

12.6 Feedback Budget and Noise

The feedback-budget and feedback-noise diagnostics (protocols above) run on the optimizer-decision proxy. Table 6 sweeps the visible window K: the learned completion beats ISOMER at every budget. Table 7 perturbs the observed selectivities; at 10% multiplicative noise the Router degrades gracefully (selectivity Q-error 1.375 → 1.418, join-optimal 92.9% → 92.2%), staying well below ISOMER (1.758) and stale (2.867).

12.7 Composition, Joins, and Plan-Shape Safety

The downstream-safety results are reported in full in the main paper: the six-estimator composition family, the FactorJoin join kernel, and the 12-configuration PostgreSQL plan-shape batch. Across all of these the calibrated marginal (OASIS,

Table 6: Feedback-budget sensitivity in the optimizer-decision proxy. All methods see only the K most recent feedback observations; OASIS uses the same fixed-width checkpoint with padding. Values are geometric means over all held-out predicates.

K	Stale QE	ISOMER QE	OASIS-noProj QE	OASIS QE	Hybrid QE	Aggressive QE	OASIS-noProj JoinOpt	OASIS JoinOpt	Hybrid JoinOpt	Hybrid choice
2	2.867	2.083	2.147	1.994	1.992	1.988	84.0%	85.4%	85.7%	ISOMER 31%, OASIS 66%, OASIS-noProj 2%, Stale 1%
4	2.867	1.797	1.924	1.675	1.665	1.658	86.7%	89.2%	89.3%	ISOMER 29%, OASIS 66%, OASIS-noProj 4%, Stale 1%
8	2.867	1.678	1.667	1.475	1.480	1.475	89.5%	91.8%	91.5%	ISOMER 31%, OASIS 59%, OASIS-noProj 8%, Stale 1%
16	2.867	1.664	1.543	1.402	1.425	1.428	91.0%	92.7%	92.2%	ISOMER 33%, OASIS 51%, OASIS-noProj 14%, Stale 1%

Table 7: Feedback-noise robustness in the optimizer-decision proxy. Noise is multiplicative log-normal perturbation applied only to observed feedback selectivities; future predicates and true selectivities remain unchanged. Values are geometric means over the configured held-out cases and noise seeds.

Noise	Stale QE	ISOMER QE	OASIS-noProj QE	OASIS QE	Hybrid QE	Aggressive QE	OASIS-noProj JoinOpt	OASIS JoinOpt	Hybrid JoinOpt
0%	2.867	1.664	1.565	1.401	1.437	1.441	90.9%	92.8%	92.2%
2%	2.867	1.677	1.568	1.408	1.447	1.441	90.9%	92.7%	92.1%
5%	2.867	1.702	1.582	1.427	1.460	1.451	90.7%	92.3%	91.8%
10%	2.867	1.758	1.620	1.474	1.496	1.487	90.3%	91.7%	91.3%

Router) is safe, whereas the raw prior (OASIS-noProj) can regress—it is actively harmful in the bilinear FactorJoin kernel and on the append-only NASA trace, which is why the projection and residual gating are part of the method. Adding the Soft candidate to the Router pool introduces no plan-shape regression: on the dense planner configurations the Router selects ISOMER and matches the deployed Hybrid, with ISOMER, OASIS, and the Router all near the same $\approx 1\%$ new-deviation rate. A plan-shape-aware gate, which could prefer hard projection over ISOMER on planner-facing cases, is left to future work.

13 Detailed Overhead Breakdown

Table 8: OASIS Runtime Overhead Breakdown (1000 trials)

Component	p50 (ms)	p95 (ms)	p99 (ms)
Feature tensorization	0.38	0.42	0.45
Model forward pass	0.66	0.72	0.78
Total correction	1.04	1.18	1.25
Statistics-format conversion	0.07	0.07	0.07
Combined correction + conversion	1.11	1.25	1.32
Feedback collection (per tuple batch)	0.12	0.18	0.23

For comparison, a column-level `ANALYZE` on a 10^7-row table costs an estimated 300–500 ms of sequential I/O. OASIS's combined latency of $\approx$1.13 ms is $\approx$300$\times$ cheaper per invocation, well within the 200 ms planning budget (desideratum **D2**).

The statistics-format conversion interface adds negligible overhead (0.066–0.073 ms with PostgreSQL-like defaults of 100 MCV entries and 10–20 residual bins). Even in the worst tested configuration (50 MCV, 50 bins), the conversion completes in 1.03 ms.

References

1. Arlitt, M.F., Williamson, C.L.: Web server workload characterization: The search for invariants. In: Proceedings of the 1996 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems. pp. 126–137 (1996). https://doi.org/10.1145/233013.233034
2. Bruno, N., Chaudhuri, S., Gravano, L.: STHoles: A multidimensional workload-aware histogram. In: Proceedings of SIGMOD. pp. 211–222 (2001)
3. Internet Traffic Archive: NASA-HTTP: Two months of HTTP logs from the Kennedy Space Center WWW server. https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html, accessed 2026-06-01
4. Markl, V., Megiddo, N., Kutsch, M., Tran, T.M., Lang, P., Raman, V.: Consistent selectivity estimation via maximum entropy. The VLDB Journal **16**(2), 169–188 (2007)
5. Park, Y., Zhong, S., Mozafari, B.: QuickSel: Quick selectivity learning with mixture models. In: Proceedings of SIGMOD. pp. 1017–1033 (2020)
6. Rathuwadu, L., Liu, G., Leckie, C., Borovica-Gajic, R.: Colse: A lightweight and robust hybrid learned model for single-table cardinality estimation using joint cdf. arXiv preprint arXiv:2512.12624 (2025). https://doi.org/10.48550/arXiv.2512.12624